

AI Image & Video Compressor

Top 10 Interview Questions & Answers - why this design, why not CNN/OpenCV, with intent notes

Q1. Walk me through this project end to end.

Answer: User uploads an image or video on the web UI. FastAPI (backend/main.py) saves it, decides image vs video by extension, and calls `compress_auto()`. The pipeline analyzes content features, classifies the file, predicts compression settings, encodes with Pillow or ffmpeg, measures real PSNR/SSIM, and retries with adjusted settings if targets are missed. The result is scored 1-10 against a fixed baseline, logged in sqlite, and returned with a download link.

Why asked: Shows system thinking: request lifecycle across frontend, backend, pipeline, DB.

Q2. What is the real difference between AI-adaptive and fixed baseline?

Answer: Fixed uses frozen settings for every file - JPEG q75 or H.264 CRF23, one pass, accepted blind. AI-adaptive measures each file (texture, edges, color, entropy), assigns a content class, predicts settings for that class, verifies with real metrics, and retries on a miss. Measured proof: photo 7233B vs 1632B at equal SSIM 0.9999; fixed JPEG even inflated a PNG by 158%.

Why asked: Proves you evaluate relatively, not in a vacuum - interviewers love baselines.

Q3. Why didn't you use a CNN or deep learning?

Answer: Deliberate engineering trade, not a gap. Budget was CPU-only with millisecond image latency and browser-playable MP4 output. A CNN quality-predictor or learned codec (ELIC/DCVC) needs GPU, slow inference, and custom decoders - surveyed in docs/RESEARCH.md as the upgrade path. The heuristic policy + measured feedback loop beats fixed on every sample at ~100x less complexity. Simple rule: never pay deep-learning cost before a cheap method fails - here it didn't fail.

Why asked: Most important answer: shows judgment over hype. Say the trade-off first, roadmap second.

Q4. Why numpy + PIL instead of OpenCV / scikit-image?

Answer: Zero extra dependency for what we need. SSIM/PSNR are ~15 lines of numpy (means, variances, covariance); features are basic array ops on a 256px thumbnail. OpenCV/scikit-image would add heavy native installs for functions we already hand-rolled in milliseconds. Minimal deps = easy Render deploy and fewer interview-time install failures. If we needed SIFT, face detection, or video optical flow, OpenCV would be the right call - we didn't.

Why asked: Shows dependency discipline: every library must earn its place.

Q5. Explain PSNR and SSIM. Why is 0.95 your target?

Answer: PSNR = $20 \cdot \log_{10}(255/\sqrt{\text{MSE}})$ in dB - pure signal error, blind to perception. SSIM compares luminance, contrast and structure from local means/variances/covariance - much closer to human vision. SSIM 0.95 ~ 95% information retained, our headline promise; below 0.85 we REJECT output. We report both because PSNR catches gross errors while SSIM catches structural damage.

Why asked: Know both formulas cold; 0.95 is a product promise, 0.85 is the floor.

Q6. CRF vs bitrate vs preset - explain like I'm five.

Answer: CRF sets quality (lower = better, we use 18-33); bitrate is whatever size that quality needs - +2 CRF $\approx$ 25% fewer bits. Preset sets encoder effort at the same CRF: slower preset = smarter motion search = smaller file, same quality. So: CRF controls how good, preset controls how hard the encoder tries. We use veryfast for latency, medium for the baseline.

Why asked: Classic video question - the +2 CRF fact earns marks.

Q7. How does the retry / parameter-adjustment loop work?

Answer: After each encode we compare measured SSIM against min_ssim and saving against target. Quality low -> quality+8 or CRF-3 (better). Saving low with quality headroom (SSIM > min+0.02) -> quality-8 or CRF+2 (smaller). Max 4 attempts, best accepted. Fixed mode gets exactly 1 round - it must stay frozen to be a fair baseline. This loop is the 'adjustment where required' requirement.

Why asked: It's gradient-free hill climbing with a quality guardrail - say that phrase.

Q8. Your fixed baseline made a PNG bigger. Isn't your comparison rigged?

Answer: No - that IS the finding, reported honestly. Fixed JPEG q75 on a tiny flat PNG adds JPEG headers and DCT noise to content PNG already stores well - 1621B became 4188B. The AI switched to WebP (924B). A rigged comparison would hide this row; ours keeps it because blind settings failing on some content is exactly why adaptation exists.

Why asked: Turns an attack into your strongest evidence. Never hide bad rows.

Q9. No VMAF - so is your video quality measurement real?

Answer: Real but a proxy, and I label it as such. We extract frames at fixed timestamps from source and output with ffmpeg and compare PSNR/SSIM - genuine measurement, no placeholders (an earlier hardcoded placeholder was deleted, see git history). Full VMAF needs the libvmaf model and per-frame scoring - roadmap, not claimed. Frame-sample SSIM correlates well for CRF comparisons.

Why asked: Honesty about proxy-vs-gold-standard beats claiming VMAF you don't run.

Q10. How would you scale this to production? What's next?

Answer: Stateless FastAPI replicas behind a queue (Celery/Redis) since encodes are CPU-bound and slow; object storage for files; Postgres instead of sqlite; progress callbacks for video (seconds-long jobs); keep-alpha WebP path for transparency; H.265/AV1 behind a flag; then learned upgrades in order: trained sklearn/CNN classifier -> Bayesian CRF search -> VMAF-guided loop -> GPU neural codecs.

Why asked: Shows you see beyond v1, in priority order, each step justified.